

Theory of Computation & Compilers Design

Lecture 2: Foundations & Computational Models

Date: 23.07.2026

Core Principle: Machine will do computations on the basis of Theory.

1. Definition: Theory of Computation (TOC)

Theory of Computation is a branch of Computer Science that deals with how efficiently a problem can be solved on a model of computation using an algorithm.

2. Foundational Elements of Theoretical Computation

Theoretical computation is built upon two foundational elements:

- **i) Principles:** The underlying rules, mathematical logic, formal laws, and proofs (e.g., Set Theory, Formal Languages).
- **ii) Concepts:** Abstract ideas and models (e.g., State transitions, Decidability, Computability, etc.).

3. Four Core Characteristics of Computational Machines

Every computational machine is formally defined by four core characteristics:

Characteristic	Description
i) Compatibility	How different machine models equate to or convert into one another.
ii) Limitations	What a specific machine model can or cannot compute/recognize.
iii) Complexity	The amount of resources (time and space/memory) a machine consumes for an input size n .
iv) Performance	How efficiently or conveniently a machine executes computation.

4. Applications of Theory of Computation

- **i)** It plays an essential and foundational role in compiler design.
- **ii)** In switching theory and the design and analysis of digital circuits, automata theory is directly applied.
- **iii)** Used in the design and formal verification/analysis of complex hardware and software systems.
- **iv)** To improve the correctness and robustness of programs, automata theory is utilized.
- **v)** To design finite state machines such as Moore and Mealy machines.

1. The Three Pillars of TOC (LAG)



2. Hierarchy to Make a Language

To make a formal language, we construct through the hierarchy: **Symbols** → **Alphabets** → **Strings** → **Language**.

- **Symbols:** Basic building blocks or smallest indivisible units of a language. Examples: digits ($0, 1$), letters ($a-z, A-Z$).
- **Alphabets (Σ):** A finite non-empty set of symbols. Denoted by Σ (sigma).
Examples: $\Sigma = \{a, b\}$, $\Sigma = \{0, 1\}$.
- **Strings (w):** A finite sequence of symbols chosen from an alphabet. Denoted by w (omega/w).
Example: If $\Sigma = \{a, b\}$ and string length = 2, the strings are $w \in \{aa, bb, ab, ba\}$.
- **Length of String ($|w|$):** The number of symbol positions in a string.
Handwritten note example: $w = ab aaba \Rightarrow |w| = 6$ (যেমন: sequence=1, length=6). An empty string has length 0 and is denoted by ϵ .
- **Language (L):** A collection of strings chosen from some Σ^* .
Analogy: All valid C-programs are strings over ASCII; the set of all syntactically valid programs forms the Language.

3. Power of Sigma (Σ) & Exponential Notation

If Σ is an alphabet, we can express the set of all strings of a certain length from that alphabet using exponential notation Σ^k , where k is the string length:

- $\Sigma^0 = \{\epsilon\}$ (string of length 0)
- $\Sigma^1 = \{a, b\}$ or $\{0, 1\}$
- $\Sigma^2 = \{aa, bb, ab, ba\}$ (length 2; total strings = $2^2 = 4$)
- $\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$ (length 3; total strings = $2^3 = 8$)
- Σ^* is the "mother of all strings" (Kleene star closure).

Finite vs. Infinite Representation:

Let $L = \{w \mid w \text{ starts with } 'a'\} = \{a, aa, aaa, ab, \dots\}$. This is an *Infinite Language*. A **Finite Automaton (FA)** provides a finite representation capable of deciding membership: strings starting with 'a' → **Yes**; strings starting with 'b' → **No**.

1. Automata Theory and Language

Automata theory deals with the definitions and structural properties of various mathematical models of computation.

Definition: An automaton is a self-operating mathematical system or abstract machine that performs a predefined sequence of operations automatically upon receiving discrete inputs.

Core Mathematical Models

1. Finite Automata (FA)
2. Context-Free Grammar (CFG) / Pushdown Automata
3. Turing Machine (TM)

Types of Automata

1. Finite Automata (FA)
2. Pushdown Automata (PDA)
3. Linear Bounded Automata (LBA)
4. Turing Machine (TM)

2. Finite Automata (FA) Representations

A Finite Automaton is an abstract computing device with discrete inputs, states, and transitions. It can be represented in three ways:

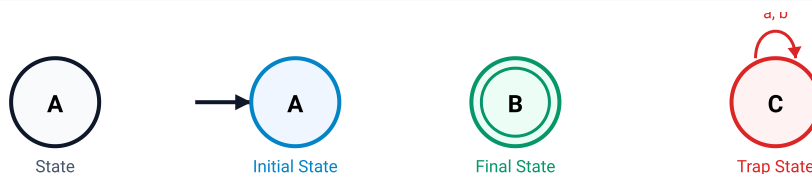
- **i) Graphical:** State Transition Diagram (directed graph with nodes as states and labelled edges).
- **ii) Tabular:** State Transition Table.
- **iii) Mathematical:** Transition function mapping (δ).

3. Formal Definition of FA (5-Tuple Specification)

A Finite Automaton is formally specified by a 5-tuple: $(Q, q_0, F, \Sigma, \delta)$, where:

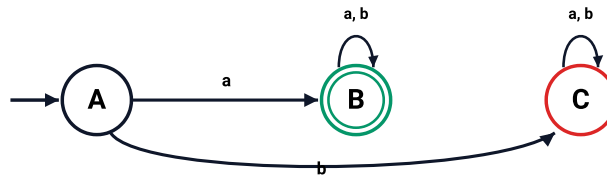
- Q = Finite set of internal states.
- $q_0 \in Q$ = Initial state (start state).
- $F \subseteq Q$ = Set of final (accepting) states.
- Σ = Finite input alphabet.
- δ = State transition function: $\delta: Q \times \Sigma \rightarrow Q$.

4. Conventions in State Diagrams



Exam Rule Note:

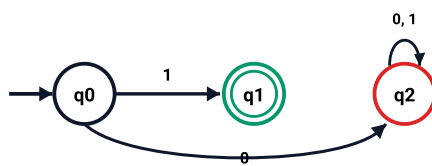
- পরীক্ষায় formal definition of FA বললে এটি tuple সহ সব লিখতে হবে।
- যদি চিত্র দেওয়া থাকে তাহলে formal definition এবং specific values লিখতে হবে।

Example 1: 3-State Finite Automaton**Formal 5-Tuple Description:**

- $Q = \{A, B, C\}$
- $q_0 = A$ (Initial state)
- $F = \{B\}$ (Final state)
- $\Sigma = \{a, b\}$ (Alphabets)
- $\delta: Q \times \Sigma \rightarrow Q$ (Transition function)

Transition Table:

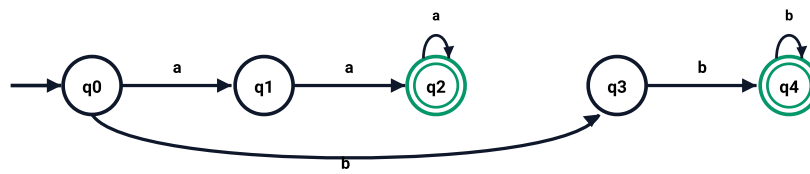
States	a	b
$\rightarrow A$	B	C
* B	B	B
C	C	C

Example 2: 3-State Automaton on Binary Alphabet $\{0, 1\}$ 

States	0	1
$\rightarrow q_0$	q_2	q_1
* q_1	q_2	q_2
q_2	q_2	q_2

$$Q = \{q_0, q_1, q_2\}, q_0 = q_0, F = \{q_1\}, \Sigma = \{0, 1\}$$

1. 5-State FA Example (Multiple Accepting States)

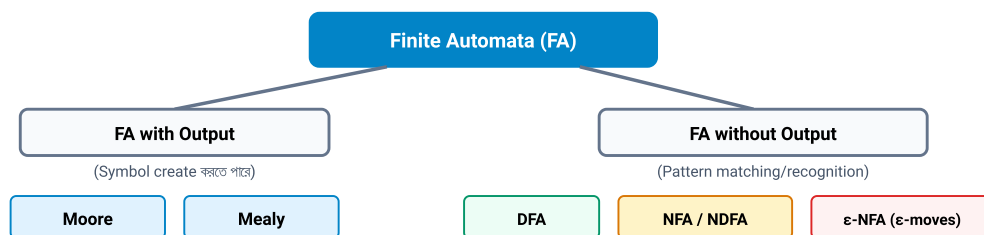


Tuple Description:

- $Q = \{q_0, q_1, q_2, q_3, q_4\}$
- $q_0 = q_{\text{start}}$
- $F = \{q_2, q_4\}$ (multiple accepting states)
- $\Sigma = \{a, b\}$
- $\delta: Q \times \Sigma \rightarrow Q$

State	a	b
$\rightarrow q_0$	q_1	q_3
q_1	q_2	q_3
$* q_2$	q_2	q_3
q_3	q_4	q_3
$* q_4$	q_4	q_3

2. Complete Classification of Finite Automata



1. Concept of Determinism

Deterministic: Deterministic means that the outcome is 100% fixed by the initial conditions, so the exact same cause will always lead to the exact same effect with no randomness.

- *Mathematical Example:* Consider a function $f(n) = n + 5$. If $n = 3$, then $f(3)$ will always equal 8. It will never result in 7 or 9.
- *Algorithmic Example:* Consider array $[4, 2, 9, 1]$. After sorting using Merge Sort, it will always result in $[1, 2, 4, 9]$.

2. Comparison: Deterministic vs. Non-Deterministic

Concept	Deterministic	Non-Deterministic
Path	Single, fixed linear path.	Multiple potential paths or branching possibilities.
Behavior	Predictable (100% certainty).	Uncertain, random or multi-choice.
Reproducibility	Perfect — re-running it will still give the exact same outcome.	Outcome or execution steps may vary across runs.



3. Formal Criteria & Rules of DFA

Bengali Rules from Notes:

- ১) প্রত্যেকটা state থেকে সবগুলো symbol নিয়ে কাজ হয়েছে কিনা ensure করতে হবে। (For every state, transitions on all symbols of Σ must exist).
- ২) সবগুলো state থেকে যেকোনো symbol শুধু একবারই বের হবে। (From any state, a symbol can trigger exactly one transition: $\delta: Q \times \Sigma \rightarrow Q$).
- ৩) Conditionally trap/dead state থাকতে পারে। (A trap state may be introduced for invalid prefixes).
- ৪) Initial state always একটি হবে কিন্তু final state এক বা একাধিক হতে পারে। (Exactly 1 start state; ≥ 0 final states).

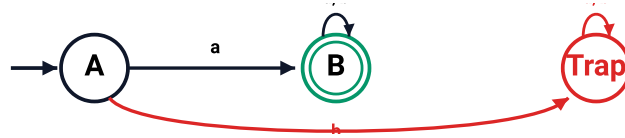
Design Sequence for Remaining Symbols: When completing missing transitions for DFA states, prioritize in this order:

1. Self Loop $\rightarrow$ 2. Back State $\rightarrow$ 3. Trap State.

Category 1: Strings Starting With a Fixed Prefix ($\Sigma = \{a, b\}$)

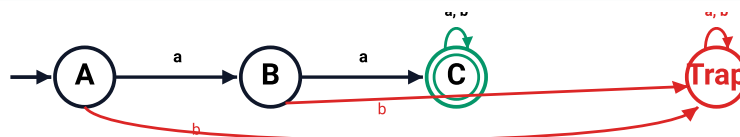
Example 1: Each string starts with 'a'

Language: $L = \{a, aa, aaa, ab, aba, \dots\}$



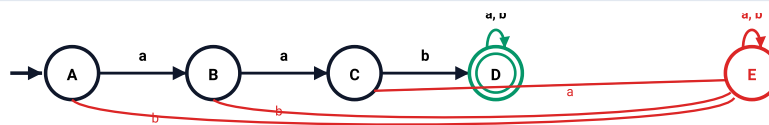
Example 2: Each string starts with 'aa'

Language: $L = \{aa, aaa, aab, aabb, aaba, \dots\}$



Example 3: Each string starts with 'aab'

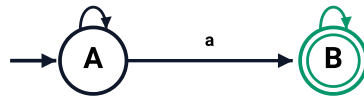
Language: $L = \{aab, aaba, aabb, aabba, \dots\}$



Category 2: Strings Containing a Specific Substring

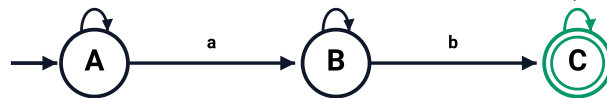
Example 4: Contains 'a' as a substring

Language: $L = \{a, aa, aaa, ba, ab, bab, \dots\}$



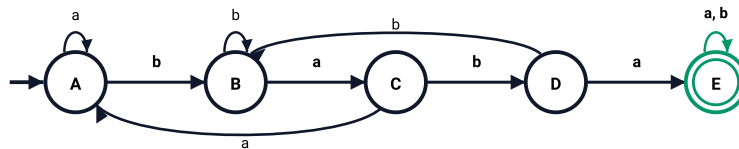
Example 5: Contains 'ab' as a substring

Language: $L = \{ab, aba, aaba, bbab, abba, \dots\}$



Example 6: Contains 'baba' as a substring

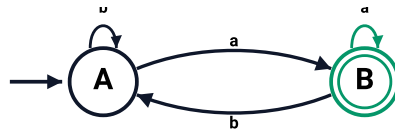
Language: $L = \{baba, ababa, ababab, abababb, babbaba, \dots\}$



Category 3: Strings Ending With a Specific Suffix

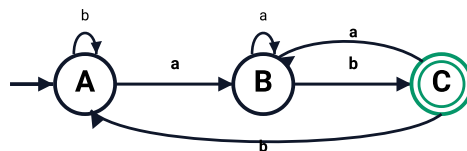
Example 7: Each string ends with 'a' ($\Sigma = \{a, b\}$)

Language: $L = \{a, aa, aaa, ba, aba, \dots\}$



Example 8: Each string ends with 'ab'

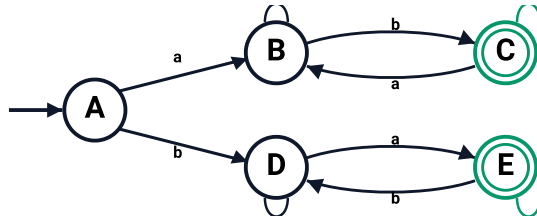
Language: $L = \{ab, aab, abbab, abab, \dots\}$



Category 4: First and Last Symbol Relations

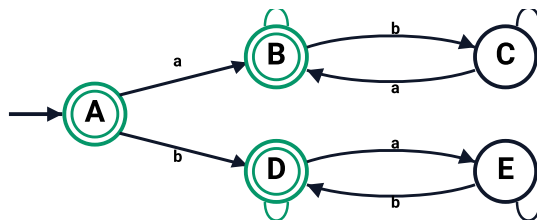
Example 9: Starts and ends with DIFFERENT symbols

Language: $L = \{w \mid w \text{ starts and ends with different symbol}\} = \{ab, ba, abb, abbb, baa, baab, \dots\}$



Example 10: Starts and ends with the SAME symbol

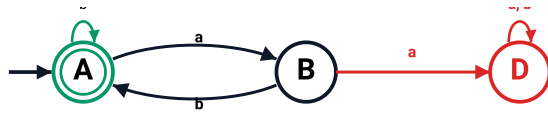
Language: $L = \{w \mid w \text{ starts and ends with same symbol}\} = \{\epsilon, a, b, aa, bb, abba, baab, \dots\}$



Category 5: Precedence & Occurrence Constraints

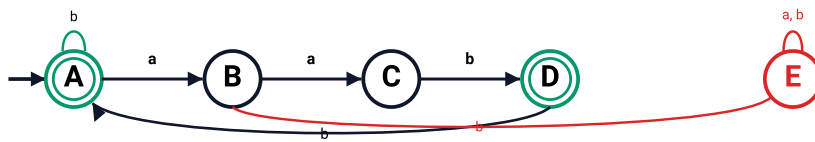
Example 11: Every 'a' in w is followed by 'b'

Language: $L = \{\epsilon, b, bb, bbb, ab, abbab, \dots\}$



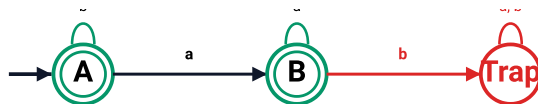
Example 12: 'a' in w is followed by 'ab'

Language: $L = \{\epsilon, b, bb, bbb, aab, aabbb, bbbab, \dots\}$



Example 13: 'a' in w is NEVER followed by 'b'

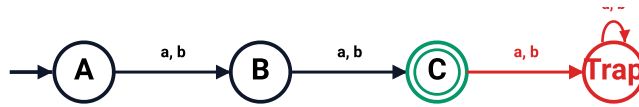
Language: $L = \{\epsilon, a, b, ba, aa, bb, aaa, bbb, baa, \dots\}$



Category 6: String Length Constraints

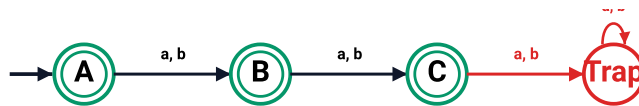
Example 1: Exact Length is 2 ($|w| = 2$)

Formula: $2^2 = 4$ combinations. **Language:** $L = \{aa, ab, ba, bb\}$ (finite language).



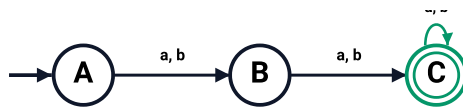
Example 2: Length at most 2 ($|w| \leq 2$) সর্বোচ্চ ২, সর্বনিম্ন ০

Language: $L = \{\epsilon, a, b, aa, bb, ab, ba\}$



Example 3: Length at least 2 ($|w| \geq 2$)

Language: $L = \{aa, bb, ab, ba, aaa, bbb, aba, abb, \dots\}$



1. Definition: DFA Minimization

DFA Minimization is the formal process of transforming a given Deterministic Finite Automaton (DFA) into an equivalent DFA that has the minimum possible number of states while recognizing the exact same regular language.

2. Why Minimize a DFA?

- **i) Reduces the Number of States:** Eliminates unreachable states from the start state and merges equivalent states into single equivalence classes, producing the minimal possible state machine.
- **ii) Time, Memory, and Performance:** Smaller DFAs result in fewer states and drastically reduced memory usage to store state transition tables. Fewer states and transitions speed up processing and lookup time when checking input strings.
- **iii) Language Equivalence and Testing:** It creates a canonical/standard, unique minimal representation for any regular language, making it straightforward to check if two different automata accept the exact same language.

3. Mathematical Equivalence Condition

Equivalence Theorem:

Two states $p, q \in Q$ are equivalent if and only if for all strings $w \in \Sigma^*$:

$$\delta(p, w) \in F \Leftrightarrow \delta(q, w) \in F$$

If there exists any string w such that $\delta(p, w) \in F$ and $\delta(q, w) \notin F$ (or vice versa), then p and q are distinguishable.

4. Step-by-Step Procedure for DFA Minimization

1. **Step 1:** Remove all states that are unreachable (inaccessible) from the initial state.
2. **Step 2:** Draw the state transition table for all reachable states.
3. **Step 3:** Mark the initial state and final states.
4. **Step 4: Partition Equivalence (Myhill-Nerode / Table-filling):**
 - **0-Equivalence (Π_0):** Partition into accepting and non-accepting states: $\Pi_0 = [F] \times [Q \setminus F]$.
 - **k-Equivalence (Π_k):** Iteratively split state groups if their transitions on any alphabet symbol land in different partitions.
 - যতক্ষণ পরপর দুইটা partition হুবহু না মিলবে ($\Pi_k = \Pi_{k-1}$), ততক্ষণ equivalence run করব।
5. **Step 5:** Reconstruct the minimized DFA where each final partition represents a single unified state.

Example 1: Minimization of 4-State DFA

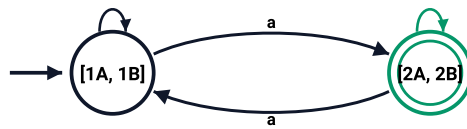
Original Transition Table:

State	a	b
→ 1A	2A	1B
1B	2A	1B
* 2A	1A	2B
* 2B	1A	2B

Step-by-Step Equivalence Partitions:

- **Step 1:** All states are reachable from 1A.
- **0-equivalence:** $[2A, 2B] [1A, 1B]$
- **1-equivalence:**
 - For $[1A, 1B]$: on 'a' → 2A, on 'b' → 1B (Identical).
 - For $[2A, 2B]$: on 'a' → 1A, on 'b' → 2B (Identical). $\Rightarrow [2A, 2B] [1A, 1B]$
- **2-equivalence:** Same as 1-equivalence. Terminate.

Resulting Minimized 2-State Automaton:



Example 2: Unreachable State Elimination

Step-1: initial state (q_0) থেকে q_3 তে কোনো পথ নেই (unreachable), তাই q_3 সম্পূর্ণ বাদ যাবে।

State	a	b
→ q_0	q_1	q_2
* q_1	q_1	q_1
* q_2	q_1	q_2

Equivalence Derivation:

- **0-equivalence:** $[q_1, q_2] [q_0]$
- **1-equivalence:**
 - For $[q_1, q_2]$: on 'a' → q_1 ; on 'b' → q_1, q_2 (both in same set).
 - $\Rightarrow [q_0] [q_1, q_2]$
- **2-equivalence:** $[q_0] [q_1, q_2]$ (Terminated).

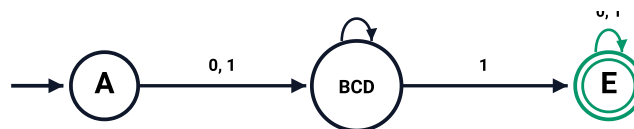
Example 3: 5-State Machine Minimization ($\{0, 1\}$)

State	0	1
$\rightarrow A$	B	D
B	C	E
C	B	E
D	C	E
* E	E	E

Derivation:

- **Step 1:** All vertices reachable.
- **0-equiv:** $[E] [A, B, C, D]$
- **1-equiv:** Test $[A, B, C, D]$:
 - A on 0 $\rightarrow$ B, on 1 $\rightarrow$ D (both non-final).
 - B, C, D on 1 $\rightarrow$ E (Final state!). $\Rightarrow [E] [A] [B, C, D]$
- **2-equiv:** $[E] [A] [B, C, D]$ (Stable).

Final Minimized Diagram (3 States):



Example 4: 4-State DFA with Distinct Partitions

State	a	b
$\rightarrow A$	B	A
B	B	C
C	B	D
* D	B	A

Derivation:

- **0-equiv:** $[D] [A, B, C]$
- **1-equiv:** On 'b': $A \rightarrow A, B \rightarrow C, C \rightarrow D$
 $\Rightarrow [D] [A, B] [C]$
- **2-equiv:** Test $[A, B]$ on 'b': $A \rightarrow A, B \rightarrow C$ (split!).
 $\Rightarrow [D] [C] [A] [B]$
- **3-equiv:** $[A] [B] [C] [D]$. All states are distinct! Already minimal.

1. Definition: Non-Deterministic Finite Automaton (NFA)

An automaton is called an NFA (Non-Deterministic Finite Automaton) when there can exist zero, one, or many possible paths/transitions for a specific input symbol from the current state to the next state.

Key Theorem: Every NFA is not a DFA, but every NFA can be formally converted into an equivalent DFA that accepts the exact same language ($L(NFA) = L(DFA)$).

2. Comparison: DFA vs. NFA

Deterministic Finite Automata (DFA)	Non-Deterministic Finite Automata (NFA)
1) Exactly one transition path for a specific input symbol.	1) Zero, one, or many paths for a specific input symbol.
2) Moves strictly require reading an input symbol (ϵ -transitions not allowed).	2) Can change state without reading input (ϵ -moves allowed).
3) Execution path is a single, predictable linear sequence.	3) Multiple branching paths (tree of possibilities).
4) Strictly follows predefined transition table. All input symbols must have transitions.	4) Explores multiple paths simultaneously. Missing transitions are implicitly rejected.
5) A string w is accepted iff its single designated path ends in a final state.	5) A string w is accepted if at least one computational branch reaches a final state.

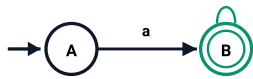
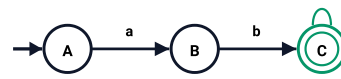
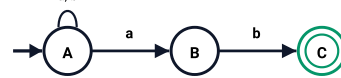
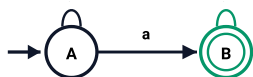
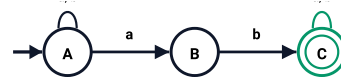
3. Formal Definition of NFA (5-Tuple Specification)

An NFA is formally defined as a 5-tuple: $(Q, q_0, F, \Sigma, \delta)$, where:

- Q = Finite set of states.
- $q_0 \in Q$ = Initial state.
- $F \subseteq Q$ = Set of final (accepting) states.
- Σ = Finite set of input symbols (alphabet).
- δ = Transition function mapping to the power set of states: $\delta: Q \times \Sigma \rightarrow 2^Q$ (or $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$).

Mathematical Mapping Note:

In DFA, $\delta: Q \times \Sigma \rightarrow Q$ (a unique state). In NFA, $\delta: Q \times \Sigma \rightarrow 2^Q$ where 2^Q is the power set (set of all subsets of Q). The return value can be $\emptyset$ (empty set/dead branch), a single state, or multiple states.

Standard NFA Design Examples ($\Sigma = \{a, b\}$)**Example 1: Starts with 'a'****Example 2: Starts with 'ab'****Example 3: Ends with 'a'****Example 4: Ends with 'ab'****Example 5: Contains 'a'****Example 6: Contains 'ab'**

1. Why Convert NFA to DFA?

While NFAs are significantly simpler and more compact to design intuitively for formal regular languages, real-world hardware and compiler lexical analyzers cannot simulate non-deterministic multi-branching in parallel without backtracking overhead. A DFA guarantees strictly linear $O(n)$ processing time.

2. Example 1: NFA to DFA (Strings Ending with 'a')

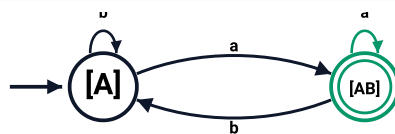
NFA Transition Table:

State	a	b
$\rightarrow A$	{A, B}	{A}
* B	$\emptyset$	$\emptyset$

DFA Table (Subset Construction):

DFA State	a	b
$\rightarrow [A]$	[AB]	[A]
* [AB]	[AB]	[A]

Converted DFA State Transition Diagram:



3. Example 2: NFA to DFA (Strings Containing 'ab')

NFA Transitions:

$$\delta(q_0, a) = \{q_0, q_1\}, \delta(q_0, b) = \{q_0\}$$

$$\delta(q_1, b) = \{q_2\}, \delta(q_2, a, b) = \{q_2\}$$

DFA State	a	b
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_0]$
$[q_0, q_1]$	$[q_0, q_1]$	$*[q_0, q_2]$
$*[q_0, q_2]$	$*[q_0, q_1, q_2]$	$*[q_0, q_2]$

Converted DFA Diagram:

